

Aqua Sphere OS: Master Requirements & Multi-Company Architecture

This document serves as the absolute single source of truth for the **Aqua Sphere OS** system architecture, data models, operational workflows, and transactional business rules. It integrates the original software blueprint, handwritten manager notes, and transcribed specifications into a unified, non-redundant technical guide optimized for AI-driven prototype generation.

1. Core Architectural Pillar: Dual-Company, Dual-Database, Single-Frontend

The absolute most critical structural requirement of Aqua Sphere OS is its multi-tenant, dual-company design. The system runs on a single, responsive frontend application but routes all operations to two completely isolated, structurally identical databases representing the two main company entities: **AquaSphere** and **Badana Industries**.

graph TD

%% User Frontend Access

subgraph UI ["Single Shared Frontend App (Mobile Responsive)"]

MM_UI["Marketing Manager Workspace (Context Toggle)"]

Owner_UI["Owner Workspace (Consolidated/Toggle Views)"]

Staff_UI["Staff Workspace (AquaSphere or Badana Portal)"]

end

%% Routing Layer

Router{"Database Context Router"}

MM_UI -->|API Session Context| Router

Owner_UI -->|API Session Context| Router

Staff_UI -->|API Session Context| Router

%% Databases

subgraph DB1 ["Database A: AquaSphere"]

AS_Cust["Customers Table"]

AS_Ord["Orders Table"]

AS_Inv["Inventory & Ledgers"]

AS_Fin["Financial Records"]

end

subgraph DB2 ["Database B: Badana Industries"]

BI_Cust["Customers Table"]

BI_Ord["Orders Table"]

```
BI_Inv["Inventory & Ledgers"]
BI_Fin["Financial Records"]
end
```

Router -->|Query Routing: AS| DB1

Router -->|Query Routing: BI| DB2

```
classDef ui fill:#e3f2fd,stroke:#0d47a1,stroke-width:2px;
classDef db fill:#f3e5f5,stroke:#4a148c,stroke-width:2px;
classDef router fill:#fff3e0,stroke:#e65100,stroke-width:2px;
class MM_UI,Owner_UI,Staff_UI ui;
class DB1,DB2 db;
class Router router;
```

1.1 Multi-Tenant Separation Rules

- **Database Isolation:** All transactional records, sales history, customer databases, credit ledgers, and raw/finished inventories must be completely segregated. Writing a transaction to AquaSphere must have zero impact on Badana Industries' state.
- **The Shared Marketing Manager (Order Desk):** The physical operator acting as the Marketing Manager manages orders for both companies simultaneously. Their user account is mapped to both databases. The UI must feature a highly visible, persistent **Company Toggle Header** (e.g., [Active Workspace: AquaSphere | Badana Industries]) allowing them to switch contexts instantly in a single click without logging out.
- **Shared and Dedicated Users:**
 - **Marketing Manager / Owner / Admin:** Can switch context between both databases dynamically.
 - **Production Manager / Accountant:** Typically assigned to a specific company workspace upon login (or granted toggle permissions based on organizational authorization).
- **System Identity:** The frontend acts as the single operational portal, while custom white-labeled customer-facing assets (like invoices and alerts) dynamically render branding matching the active database context (AquaSphere or Badana Industries).

2. System Role Hierarchy & Permissions (5-Role Stack)

The system enforces a 5-role operational stack split across supervisory verification and day-to-day execution.

2.1 Role Hierarchy Diagram

graph TD

Owner["Owner (Master Control / Multi-Company Viewer)"]
 Admin["Admin (Supervisor / View-Only Closer)"]
 PM["Production Manager (PM)"]
 Accountant["Accountant (Finance & Auditing)"]
 MM["Marketing Manager (Order Desk)"]

Owner <--> |Strategic Overviews| Admin
 Admin --> |Supervises & Locks| PM
 PM --> |Co-ordinates Production Logs| Accountant
 Accountant --> |Verifies Credit & Collections| MM

```
classDef supervisor fill:#d4edda,stroke:#28a745,stroke-width:2px;
classDef ops fill:#f8d7da,stroke:#dc3545,stroke-width:2px;
class Owner,Admin supervisor;
class PM,Accountant,MM ops;
```

2.2 Detailed Permission & Security Matrix

Role	Access Level	Primary Focus	Critical Operations & Restrictive Boundaries
Owner	Full System Admin	Analytics, Security, Overrides	Full Control: Can read and write to all elements across both databases. The <i>only</i> role authorized to delete customer records, manually override inventory counts, modify passwords, and update website parameters.
Admin	Supervisor (View-Only)	Verification & Daily Closing	Strictly View-Only: Cannot insert/edit daily transactions, cannot view net profit calculations, and cannot view

			raw material cost metrics. Can view stock counts, production totals, and orders. Admin manually closes the daily ledger, locking editing capabilities.
Production Manager (PM)	Operational Input	Production Lines & Raw Stocks	Enters daily finished production counts (0.5L and 1.5L packs) and logs bottle breakages during line runs. Closes their section at the end of each day. No access to financial, sales, or customer records.
Accountant	Financial Auditor	cash flows, Expenses, ledgers	Enters operational expenses (must upload receipt photos) and spot sales. Generates invoices and manages accounts payable/receivable. Strictly blocked from direct inventory view/adjustment.
Marketing Manager	Operational Sales	CRM & Fast Order Entry	Enters customer records, handles incoming calls, inputs orders (0.5L , 1.5L , 19L), tracks

			deliveries, logs payment methods/statuses. Blocked from deleting customer records.
--	--	--	--

3. Role-Specific Workflow Specifications

3.1 Admin Close-of-Day Workflow

At the end of each day, the Admin must freeze editing capabilities for that operational day by cross-verifying active records.

1. **WhatsApp Cross-Verification:** The Admin checks external WhatsApp delivery channels, comparing orders and cash collected with the live system portal metrics.
2. **Portal Check:** The Admin reviews:
 - Total stock numbers on hand
 - Daily production outputs (**0.5L** and **1.5L** packages)
 - All customer orders and cash balances
3. **Manual Close Action:** Once satisfied, the Admin clicks the **"Close Daily Ledger"** button. This permanently locks the date's transactions. After this, no entries can be added or edited by any role except the Owner.

3.2 Marketing Manager Order Entry & Smart CRM

The order entry module is designed to execute in under **20 seconds** during active phone calls.

- **Fast Customer Lookup:** The operator searches exclusively by Phone Number, Name, or Customer ID. Instantly, the UI displays the customer profile, outstanding balance, current **19L** bottle holdings, last delivery date, and average monthly orders.
- **"Smart Add" Customer Flow:** If a customer record is not found during a search, clicking the "Add New Customer" button prompts a modal directly within the active order workflow to register:
 - Primary contact details (Phone must be unique)
 - GPS/Google Maps Location Pin
 - **Home/Building Front Picture:** An uploaded photo of the customer's storefront or house exterior to aid delivery drivers in rapid identification.
- **Daily Cleanliness Policy:** All orders placed on the current date must be closed or marked complete by end-of-day. Any un-cleared orders automatically escalate to the Owner's and Admin's dashboard with warning indicators.

3.3 Accountant Daily Financial Inputs

- **Mandatory Expense Receipts:** To input a daily operational expense (e.g., fuel, plant rent, maintenance, salaries), the Accountant **must upload a picture** of the physical bill or receipt. Text-only entries are disallowed. Alternatively, they can upload a picture of a hand-written paper ledger sheet and subsequently transcribe the details.
- **Spot Sales (Counter Sales):** Walk-in customers frequently bring non-standard personal containers. The Accountant records:
 - Total water sold (quantified in Liters)
 - Number of caps issued
 - Cash collected
 - Assigned credit limits (if applicable)

4. Blowing Machine Division Portals

The manufacturing of empty bottles via the Blowing Machine operates inside its own distinct workflow containing **five segmented portals**.

graph TD

BM[Blowing Machine Access Point] --> Selection{Select Workspace Division}

Selection -->|Badana| BadanaTrack[Badana Blowing Portal]

Selection -->|Aqua Sphere| AquaTrack[Aqua Sphere Blowing Portal]

subgraph BadanaTrack [Badana Division]

B_Prod[Production: Factory to Warehouse]

B_Sale[Sales: Direct Factory/Warehouse]

B_Preform[Preform Weights Tracked]

end

subgraph AquaTrack [Aqua Sphere Division]

A_Prod[Production: Factory to Warehouse]

A_Sale[Sales: Direct Factory/Warehouse]

A_Preform[Preform Weights Tracked]

end

style BadanaTrack fill:#fffde7,stroke:#fbc02d,stroke-width:1px

style AquaTrack fill:#e8f5e9,stroke:#388e3c,stroke-width:1px

- **Preform Composition Tracking:** The Blowing Machine tracks two material grades (Pure Preforms and Mix Preforms) with localized unit weights:
 - **Pure Preforms:** $1.5L = 30\text{ g}$ per bottle | $0.5L = 15\text{ g}$ per bottle.
 - **Mixed Preforms:** $1.5L = 27\text{ g}$ per bottle | $0.5L = 13\text{ g}$ per bottle.
- **Stock Movements:** Tracks inventory transfers directly moving from the **Blowing Factory**

to the **Primary Warehouse**. Sales can occur directly from either storage site.

5. Inventory, Minerals, & Production Calculations

All calculations utilize the exact ledger-based transaction method. No stock levels are directly editable fields; they are derived dynamically from transaction history.

5.1 Mineral Treatment Formulations

Mineral inventories are tracked in kilograms (**kg**). The chemical treatment formulation is:

$$1 \text{ Mineral Set} = 2 \text{ kg Calcium} + 1 \text{ kg Magnesium} + 0.5 \text{ kg Sodium}$$

-

$$1 \text{ Mineral Set treats exactly } 15,140 \text{ Liters of raw water}$$

Decimal Precision Mandate: Mineral consumption must be calculated and deducted from inventories using exact decimal fractions. Rounding must not occur at transaction runtime to prevent cumulative calculation drift over thousands of entries.

5.2 Water Allocation & Production Ratios

- **Finished Product Water Consumption:**
 - One **19L** bottle delivered $\approx 23 \text{ Liters}$ of mineral water.
 - One **0.5L** PET pack (**12 bottles/pat**) produced $\approx 9 \text{ Liters}$ of raw water.
 - One **1.5L** PET pack (**6 bottles/pat**) produced $\approx 12 \text{ Liters}$ of raw water.
- **Consumable Unit Weights per Pack (Pat):**
 - **0.5L** packs require exactly **6.72 g** of labels per pack.
 - **1.5L** packs require exactly **7.86 g** (recorded as **7.8 g** to **7.86 g**) of labels per pack.
 - *Note: Labels for different sizes are tracked on separate lines.*

5.3 Low-Stock Dashboard Warnings (Reorder Thresholds)

When raw inventory falls to or below the following levels, a non-blocking warning indicator lights up on the Owner and PM dashboards:

Raw Material	Dashboard Warning Threshold
--------------	-----------------------------

Sodium	3 kg
Calcium	10 kg
Magnesium	5 kg
1.5L Empty Bottles	6,000 units
0.5L Empty Bottles	6,000 units
1.5L Labels	15 kg
0.5L Labels	10 kg
Shrink Wrap	10 kg
Small Caps	6,000 units
Large Caps (19L)	500 units
19L Empty Bottles	50 units

6. Credit Limits, CRM Alerts, & Outbound Notifications

- **Consistent Balance Naming:** To prevent confusion, the terms "**Customer Outstanding**" (on sales workspaces) and "**Credits**" (on accounting ledgers) reference the exact same system variable and display matching balances.
- **The Soft-Block Philosophy:** The system never blocks an operator mid-call. If a customer's balance exceeds their credit limit, or returns more empty bottles than historically held, the system prompts a warning but allows the transaction to proceed upon confirmation.
- **Automated Outbound Alerts:**
 - **Credit Breach Scenario:** If a customer's credit limit is 5,000 PKR and their unpaid balance reaches 6,000 PKR, the system flags the order and displays a single-button option to auto-generate a WhatsApp billing notice.
 - **Inactivity Scenario:** If a customer goes 1 week without placing an order, the CRM

automatically highlights them on the Marketing Manager's dashboard as "Requires Follow-Up."

7. 19L Reusable Bottle Ledger

19L bottles are treated as capital equipment. They circulate in a closed loop, meaning there is no finished goods inventory for **19L** bottles (filling occurs during delivery).

stateDiagram-v2

[*] --> StartTransaction : Deliver / Return Order Processed

```
state TransactionSynchronization {
    [*] --> WritePurchaseLedger : Record Financial Asset Transaction
    [*] --> UpdatePhysicalInventory : Adjust Customer vs. Factory Stock counts
    WritePurchaseLedger --> ReconcileVerification
    UpdatePhysicalInventory --> ReconcileVerification
    ReconcileVerification --> CommittedState
}
```

CommittedState --> [*] : Transaction Successfully Closed

7.1 Reconciled Balance Rule

The system must maintain strict mathematical alignment across five variables calculated from the append-only bottle transaction history:

$$\text{Total Owned} = \text{At Factory} + \text{With Customers} + \text{Broken}$$

- **Lost Bottles:** Written-off or lost bottles are subtracted from the **Total Owned** balance.
- **Return Split:** Empty bottle returns must be recorded as either **Good Returns** (clean, reusable) or **Broken Returns** (written off as damaged).

8. Integrated Data Model

Below is the database schema mapping the dynamic transaction entities. For the multi-company architecture, this schema is deployed into **two separate database instances** (or partitioned cleanly with a strict tenant_id/company_context tenant isolation strategy at the database router level).

Customer

- id (Primary Key)
- phone (Unique string - primary search key)
- name (String)
- address (Text)
- mapsLocation (String link/coordinates)
- homePictureUrl (String - image path of building exterior)
- customerType (Enum: Home, Restaurant, Shop, Distributor)
- creditLimit (Decimal, 0 = unlimited)
- creditDuration (Interval/Int, default = 1 month)
- defaultPrice (Decimal, optional)
- remarks (Text)

Order

- id (Primary Key)
- customerId (Foreign Key)
- orderType (Enum: 19L, PET)
- deliveryStatus (Enum: pending, partial, delivered)
- paymentStatus (Enum: unpaid, partial, paid)
- expectedDelivery (Date)
- remarks (Text)

Delivery

- id (Primary Key)
- orderId (Foreign Key)
- deliveredAt (Timestamp)
- qtyDelivered (Int)
- bottlesReturnedGood (Int - 19L only)
- bottlesReturnedBroken (Int - 19L only)
- remarks (Text)

Payment

- id (Primary Key)
- orderId (Foreign Key, Nullable)
- customerId (Foreign Key)
- amount (Decimal)
- method (Enum: cash, bank, mobile_wallet, other)
- receivedAt (Timestamp)

BottleTransaction

- id (Primary Key)
- customerId (Foreign Key, Nullable - Null indicates factory adjustment)
- txnType (Enum: delivered_to_customer, returned_good, returned_broken, lost,

- purchased_new, factory_adjustment)
- qty (Int)
- refDeliveryId (Foreign Key, Nullable)
- note (Text)
- createdAt (Timestamp)

InventoryTransaction

- id (Primary Key)
- itemId (Foreign Key)
- direction (Enum: IN, OUT)
- qty (Decimal - Supports precise decimal fractions)
- refType (Enum: purchase, production_consume, production_output, sale, adjustment)
- refId (String - UUID mapping to transaction cause)

ProductionBatch

- id (Primary Key)
- productionDate (Date)
- qty05LProduced (Int - number of packs/pats)
- qty15LProduced (Int - number of packs/pats)
- mineralSetsConsumed (Decimal - high precision)